Exercise 11 - 26.01./27.01.2026

Learning goals

Today we want to repeat a few things and get hands on experience with an Angular **guard**, the most common interceptor mechanism to protect logged in screens / routes for UX convenience and perceived security.

We also want to get some hands-on experience creating and modifying a basic PWA with Angular, adding @angular/pwa via schematics.

Practical Exercises

Your existing Angular application should have multiple routed components: including the DashboardComponent accessible via /dashboard and the Login(Form)Component accessible via /login. Until now, no matter if you were already logged in or not, you could access the dashboard.

1) Create an **AuthGuard** (***ng generate guard auth***) that implements the ***CanActivateFn*** interface and allows you accessing the /dashboard ***route*** of an Angular app only if you are logged in. If not, the router shall redirect to /login.
   The login status you should have implemented already in a global singleton ***AppStateService***. Your header should display already information about login status and a logout button.
   Info: The route guard only gets triggered on initial loading of the route.
   Therefore the guard will not help redirecting automatically on logout if you accessed the dashboard in a logged in state.
   Still route guards are a great concept to initially protect „logged in“ screens or screens that should be accessible only for e.g. „admin“ users.
   Remember: frontend security like this is only a UX improvement, not a real security mechanism! All frontend code is public exposed to the browser and could be modified.
   Revisit Lecture 5 or https://angular.dev/guide/routing/common-router-tasks#preventing-unauthorized-access

Hint: the AuthGuard in modern Angular (e.g. v20) is implemented as a function. In order to inject dependencies like an AppStateService or the Router inside the AuthGuard file you can internally use this syntax:

```
const appStateService = inject(AppStateService);
const router = inject(Router);
```

2) **Add PWA capabilities** to your app using an Angular schematic to install PWA content. If you have an Angular v20 app, you need to call ***ng add @angular/pwa@20*** at root level. If successful, you will notice the minimum PWA functionalities to be created.

Hint: This schematic (code generating script) command is especially error-prone if versions do not match exactly in package.json. Maybe npm install with - - force flag is needed.

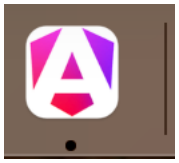
Which existing files are modified? Which ones are added? Ensure all are added to GIT version control to be able to commit and push them to GitLab.

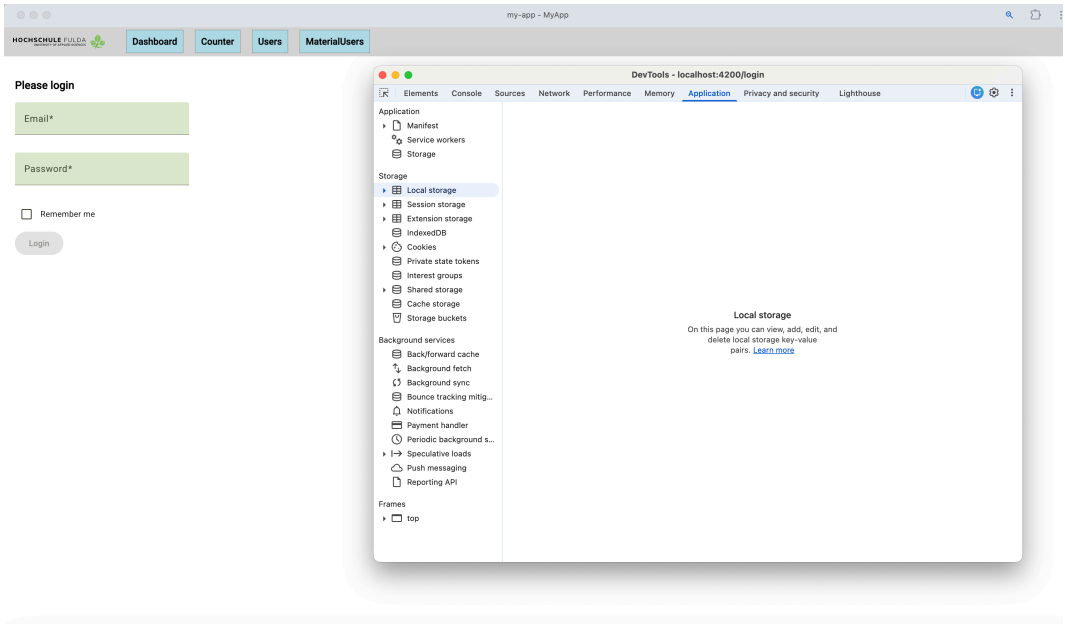
(You can answer in a Markdown .md file that you can commit to your repo or just write it down for yourself.)

Run the angular application and you should be able to „install“ the app from localhost on your desktop as an icon. Alternatively to localhost you would have to host your website using HTTPS.
You can open the chrome browser dev tools and check the „Application“ Tab to see the detection as a PWA works. You can simulate here e.g. in „Service workers“ that the app would go offline (webserver not reachable). Toggle this and reload the page.

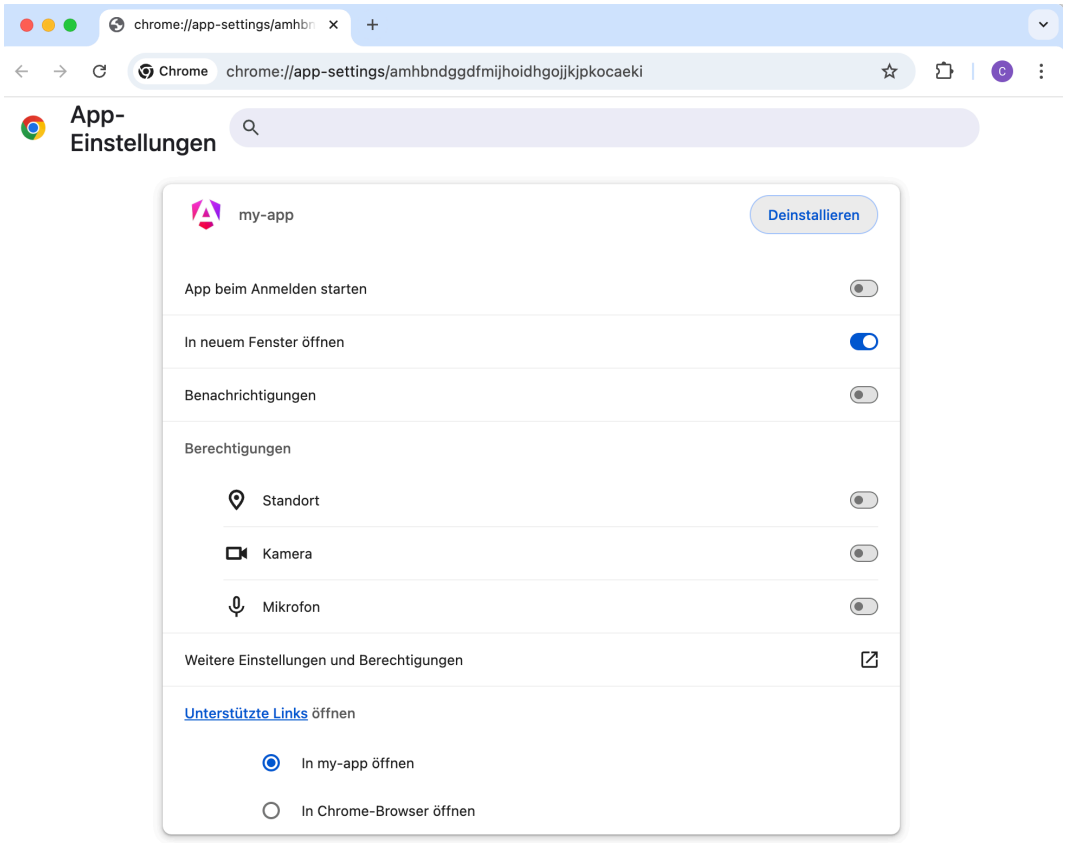
The „installed“ PWA uses the browser engine internally to run the application. Therefore you can make use of dev tools here as well.

It looks like this on my Mac:



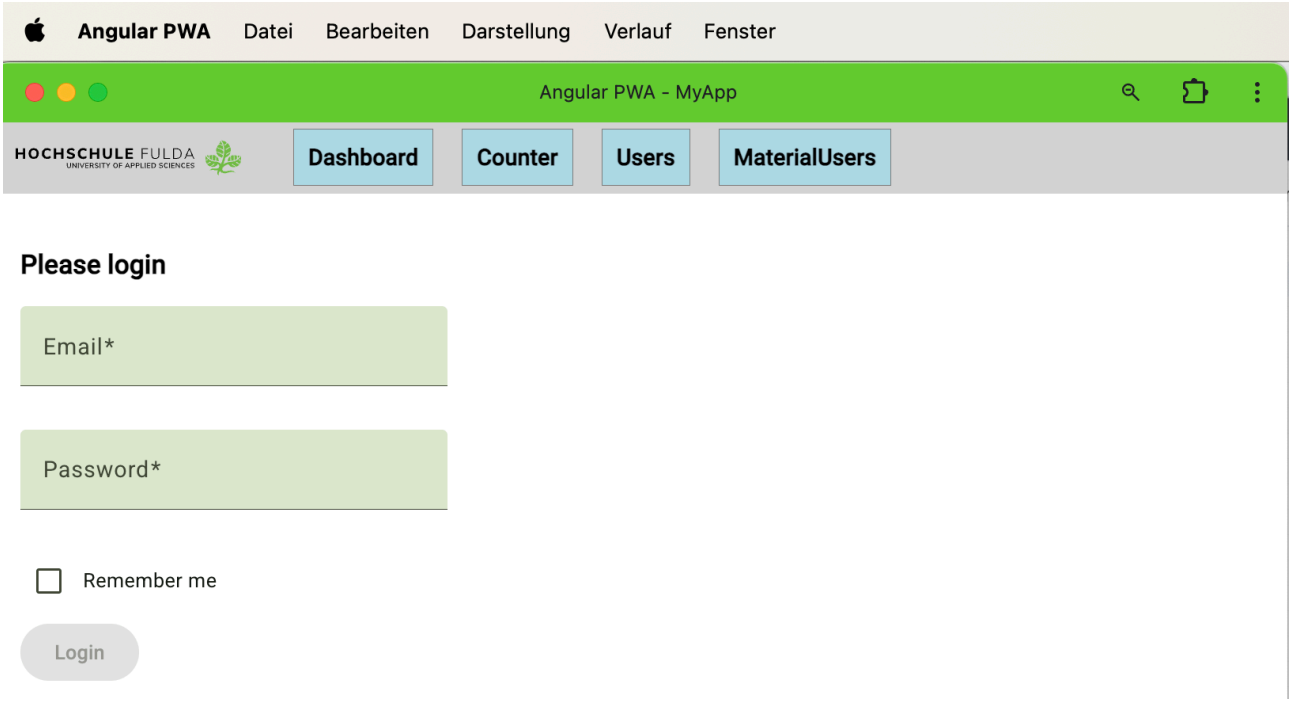


To get rid of that installed app, you can go to „settings“ and deinstall it.



3) Now modify the existing PWA and rename the application to „Angular PWA“. Change also the „theme_color“ to be HS Fulda green (#66cc33). You will need to de-install and install again for the change to be effective.

Result might look like this, note the name changed at 2 places:



4) To make full use of „offline capabilities“ it would be nice if the PWA is not only capable of caching frontend data (like asset files) but also cache backend responses for the /users request from our Node ExpressJS backend.

In order to achieve this, you have to modify the manifest file.

Add this below „assetsGroups“ and modify to match your backend URL:

```
"dataGroups": [
  {
    "name": "api-calls",
    "urls": [
      „http(s)://my-backend.com/**“
    ],
    "cacheConfig": {
      "maxSize": 100,
      "maxAge": "1d",
      "strategy": "freshness"
    }
  }
]
```

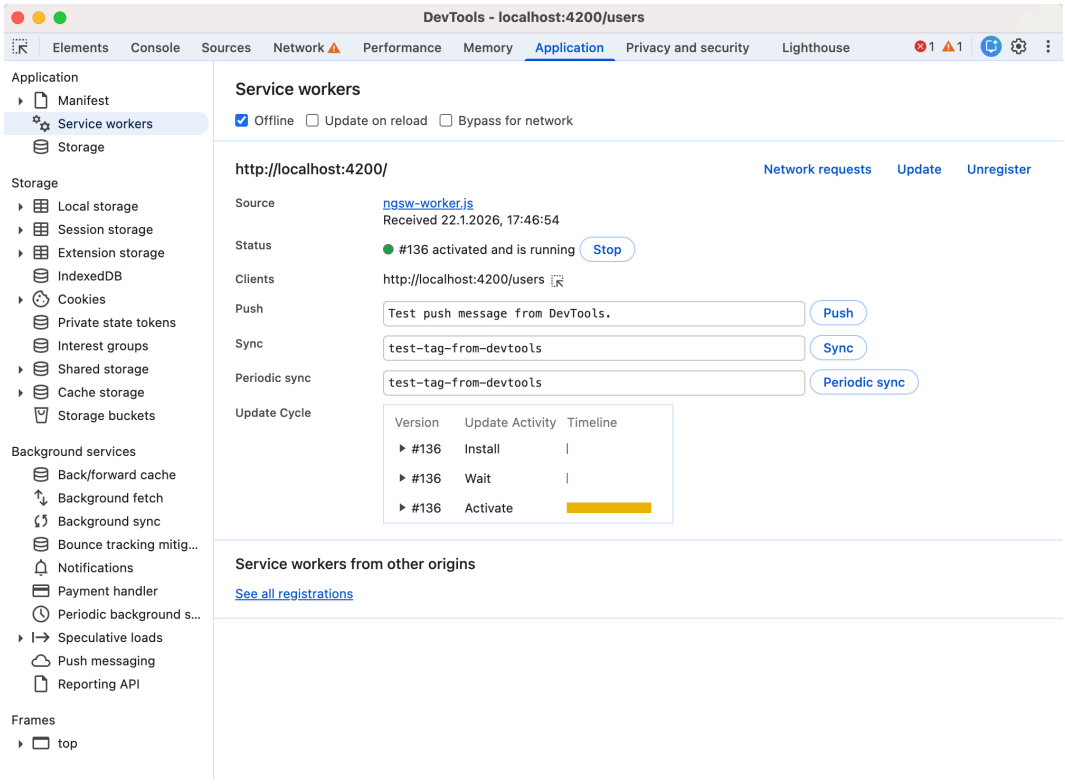
You can fine-tune how many cache results to keep, how long cache might be valid and strategies like „performance“ (cache-first) or „freshness“ (network first).

Now what is problematic, is that you do not see the service-worker JS file in your browser network. This is due to the code that prevents the SW to be activated in „dev“ server mode, which is the case if you start the Angular app via „ng serve“.

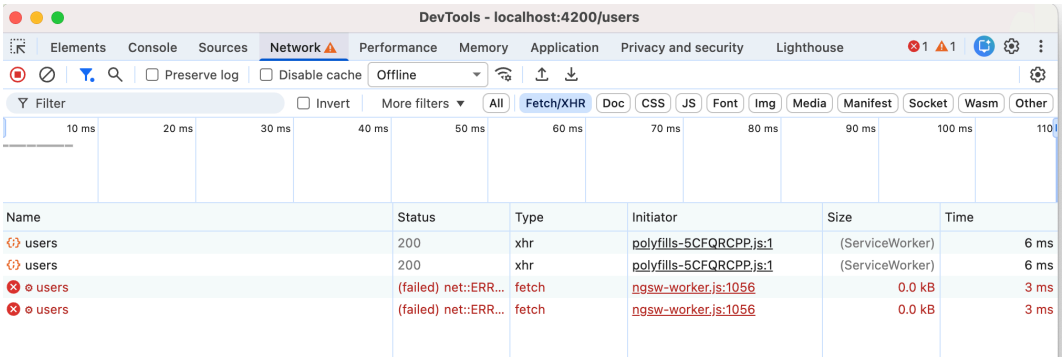
To test your PWA with the active service worker, you need a production build: „ng build“ plus serving the created static files (in /dist = distribution folder) with an externalized server, e.g. via "npx http-server -p 4200 dist/my-app/browser“. Now you see generated JS code that holds the service worker functionality.

If you try it out and everything is fine you can debug the network tab and see data is derived from the SW cache (even when toggling „offline“ mode in SW Application settings).

Result looks like this once the SW is active:



Users will be fetched from cache (if done earlier before being offline) and just the „refresh“ request for new data will fail. You can also simulate this by stopping your ExpressJS backend and toggling the PWA to be not offline anymore. Result should be the same.



More info about Service-Worker JS API can be found here:

https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API

How can you be sure the Service-Worker JS API is supported by all browsers? Remember a website we used in earlier lectures to check whether a modern CSS property was supported. That is the same website that provides reliable data for JS support.

Info: In order to prevent a white-screen outage of your website, you usually follow the pattern „do not break the web“ and ensure via javascript, that the service worker api is existing in the browser environment where it gets executed. Service worker logic should be an optional enhancement of the website and exceptions must be caught.